5 - 上下文无关文法

2023.8.19 周奇凤

上下文无关文法的形式定义

定义：上下文无关文法(CFG, Context-Free Grammar，简称文法) G是一个四元组，$G=(V,T,P,S)$，其中：

- V：变元的有穷集，变元也称为非终结符或语法范畴
- T：终结符的有穷集（即字母表），且$V \cap T = \phi$
- P：产生式的有穷集，每个产生式包括：
  - ✓ 一个变元，称为产生式的头(head)或左部；
  - ✓ 一个产生式符号→，读作定义为；
  - ✓ 一个$(V \cup T)^*$中的符号串，称为体(body)或右部（由变元和终结符组成的字符串）
- S：初始符号$S \in V$，表示文法开始的地方

如果有多个A的产生式：$A \to \alpha_1, A \to \alpha_2, ..., A \to \alpha_n$，可简写为：$A \to \alpha_1 | \alpha_2 | ... | \alpha_n$

文法中变元A的全体产生式，称为A产生式

上下文无关文法的形式定义

字符使用的一般约定：

- 变元：$A, B, C, \ldots$
- 终结符：$0, 1, a, b, c, \ldots$
- 终结符或非终结符：$\ldots, X, Y, Z$
- 终结符构成的串：$\ldots, w, x, y, z$
- 终结符或非终结符组成的串：$\alpha, \beta, \gamma, \ldots$

文法示例：回文与回文语言

定义：如果字符串$w \in \Sigma^*$满足$w = w^R$，则称字符串w为**回文**。

定义：如果语言L的字符串都是回文，则称L为**回文语言**。

$$L = \{w \in \Sigma^* | w = w^R\}$$

例：ε，010，0000，radar，drawward…

如何用上下文无关文法表示字母表$\Sigma = \{0,1\}$上的回文语言

$$L_{\text{pal}} = \{w \in \{0,1\}^* | w = w^R\}?$$

可使用递归的方式定义该语言：

- ε，0，1都是回文
- 若w是回文，则$0w0$，$1w1$也是回文

故，可以表示为$G = (\{A\}, \{0,1\}, \{A \to \varepsilon | 0 | 1 | 0A0 | 1A1\}, A)$

归约&派生

非形式定义：

- 从字符串到文法变元的分析过程，称为**归约**(reduction)；
- 从文法变元到字符串的分析过程，称为**派生**(derivation)。

形式定义：

若上下文无关文法$G=(V,T,P,S)$，设$\alpha,\beta,\gamma\in(V\cup T)^*$，$A\in V$，$A\rightarrow\gamma\in P$，那么称在$G$中由$\alpha A\beta$可以**派生**出$\alpha\gamma\beta$，记为$\alpha A\beta\underset{G}{\Rightarrow}\alpha\gamma\beta$；相应的，称$\alpha\gamma\beta$可以**归约**为$\alpha A\beta$。

- 若语境中G是已知的，可以省略不写。
- 若α_1通过零步或者多步派生可以得到α_m，那么可以记为$\alpha_1\overset{*}{\Rightarrow}\alpha_m$

归约&派生

用算术表达式文法$G_{exp} = (\{E, I\}, \{a, b, 0, 1, +, *, (,)\}, P, E)$，其中产生式集$P$中有下列10条产生式。请写出$a*(a+b00)$的归约和派生过程。

1. $E \to I$
2. $E \to \mathrm{E} + \mathrm{E}$
3. $E \to E * E$
4. $E \to (E)$
5. $I \to a$
6. $I \to b$
7. $I \to Ia$
8. $I \to Ib$
9. $I \to I0$
10. $I \to I1$

归约：

利用5，6：$I*(I+I00)$

利用9：$I*(I+I0)$

利用9：$I*(I+I)$

利用1：$E*(E+E)$

利用2：$E*(E)$

利用4：$E*E$

利用3：E

派生：

$E \Rightarrow E*E \Rightarrow E*(E) \Rightarrow I*(E) \Rightarrow I*(E+E) \Rightarrow I*(E+I) \Rightarrow I*(I+I) \Rightarrow I*(a+I) \Rightarrow a*(a+I) \Rightarrow a*(a+I0) \Rightarrow a*(a+I00) \Rightarrow a*(a+b00)$

最左派生&最右派生

为限制派生的随意性，定义只替换符号串中最左边变元的派生过程，称为**最左派生**，记为$\underset{lm}{\Rightarrow}$；只替换最右的，称为**最右派生**，记为$\underset{rm}{\Rightarrow}$。

任何派生都有等价的最左派生和最右派生。

例：表达式$a*(a+a)$在G_{exp}中的最左、最右派生为：

1. $E \to I$
2. $E \to \mathrm{E}+\mathrm{E}$
3. $E \to E*E$
4. $E \to (E)$
5. $I \to a$
6. $I \to b$
7. $I \to Ia$
8. $I \to Ib$
9. $I \to I0$
10. $I \to I1$

最左派生：

$$
\begin{aligned}
E &\underset{lm}{\Rightarrow} E*E \underset{lm}{\Rightarrow} I*E \\
&\underset{lm}{\Rightarrow} a*E \underset{lm}{\Rightarrow} a*(E) \\
&\underset{lm}{\Rightarrow} a*(E+E) \\
&\underset{lm}{\Rightarrow} a*(I+E) \\
&\underset{lm}{\Rightarrow} a*(a+E) \\
&\underset{lm}{\Rightarrow} a*(a+I) \\
&\underset{lm}{\Rightarrow} a*(a+a)
\end{aligned}
$$

最右派生：

$$
\begin{aligned}
E &\underset{rm}{\Rightarrow} E*E \underset{rm}{\Rightarrow} E*(E) \\
&\underset{rm}{\Rightarrow} E*(E+E) \\
&\underset{rm}{\Rightarrow} E*(E+I) \\
&\underset{rm}{\Rightarrow} E*(E+a) \\
&\underset{rm}{\Rightarrow} E*(I+a) \\
&\underset{rm}{\Rightarrow} E*(a+a) \\
&\underset{rm}{\Rightarrow} I*(a+a) \\
&\underset{rm}{\Rightarrow} a*(a+a)
\end{aligned}
$$

上下文无关语言

上下文无关文法$G=(V,T,P,S)$的语言定义为 $L(\boldsymbol{G})=\left\{w \middle| w \in T^{*},\ S \overset{*}{\Rightarrow} w\right\}$

即符号串w在$L(\boldsymbol{G})$中要满足:

1. w仅由终结符组成;
2. 初始符号S能派生出w。

如果语言L是某个CFG G定义的语言，即$L=L(\boldsymbol{G})$，则称L为上下文无关语言(CFL, Context-Free Language)。

上下文无关是指在文法派生的每一步$\alpha A\beta \Rightarrow \alpha\gamma\beta$，符号串$\gamma$仅根据$A$的产生式派生，而无需依赖$A$的上下文$\alpha$和$\beta$。

如果有两个文法CFG G_1和CFG G_2，满足$L(\boldsymbol{G_1})=L(\boldsymbol{G_2})$，则称$G_1$和$G_2$是等价的。

句型

定义：若上下文无关文法$G=(V,T,P,S)$，初始符号S派生出来的符号串称为G的**句型**，即$\alpha \in (V \cup T)^*$且$S \overset{*}{\Rightarrow} \alpha$。

若α是由S最左派生得到的，称α为**左句型**；

若α是由S最右派生得到的，称α为**右句型**。

只含有终结符的句型称为G的**句子**。

故$L(G)$就是文法G全部的句子。

语法分析树

若CFG $G=(V,T,P,S)$的**语法分析树**(parse tree)，也称为派生树(derivation tree)，其形式定义为：

- 每个内结点的标记是V中的变元符号
- 每个叶结点的标记是$V \cup T \cup \{\varepsilon\}$中的符号
- 如果内结点的标记是A，其子节点从左至右分别为$X_1, X_2, \ldots, X_n$，那么，$A \rightarrow X_1 X_2 \ldots X_n$是$P$的一个产生式；若某个$X_i$为 ε，则X_i一定是A的唯一的子结点，且$A \rightarrow \varepsilon$是一个产生式。

语法分析树的全部叶节点从左到右连接起来，称为该树的**产物**(yield) 或**结果**。如果树根节点是初始符号S，叶节点是终结符，那么该树的产物属于$L(G)$。

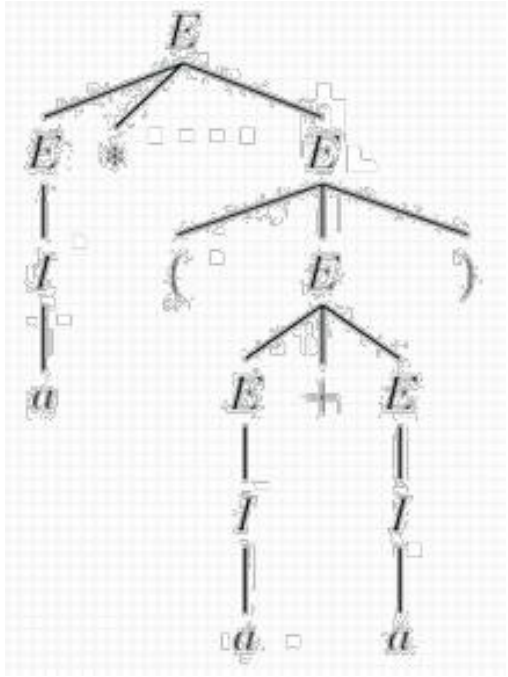
语法分析树中标记为A的内节点及其全部子孙节点构成的子树，称为A**子树**。

语法分析树

定理：若CFG $G=(V,T,P,S)$且$A\in V$，那么文法G中$A\overset{*}{\Rightarrow}\alpha$，当且仅当在文法$G$中存在以$A$为根节点，产物为$\alpha$的语法分析树。（语法分析树与派生具有等价性）

例：算术表达式文法G_{exp}推导算术表达式$a*(a+a)$的过程：

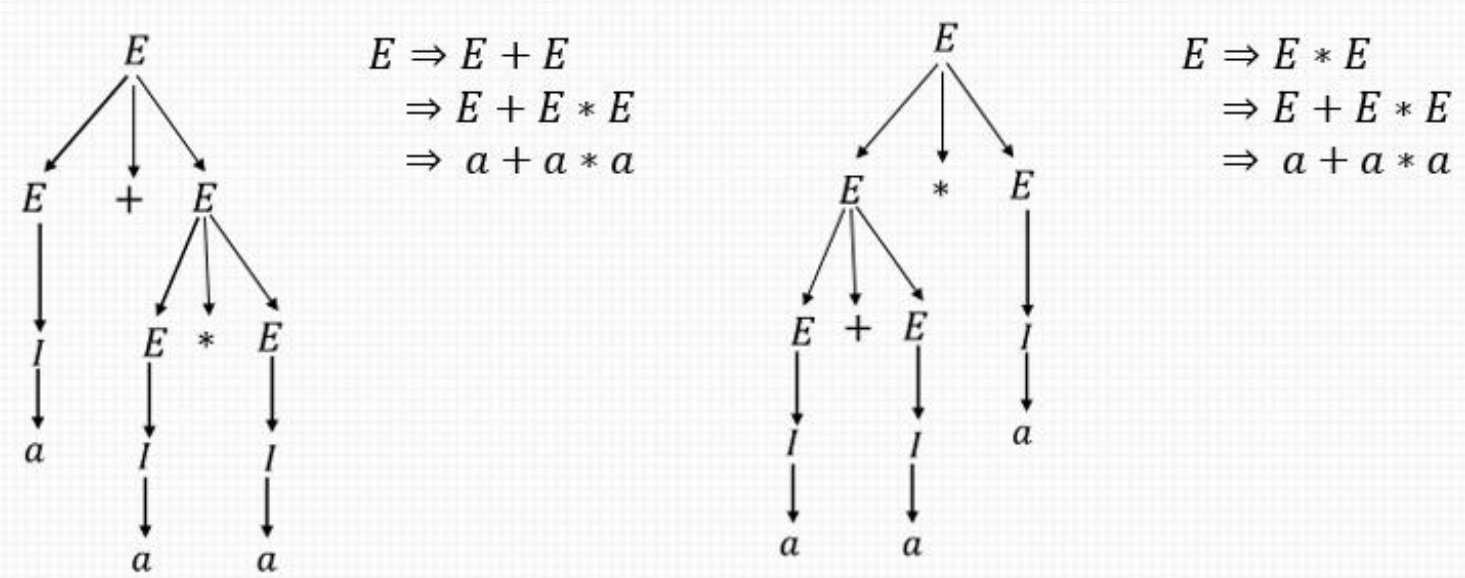
1. $E \to I$
2. $E \to \mathrm{E}+\mathrm{E}$
3. $E \to E*E$
4. $E \to (E)$
5. $I \to a$
6. $I \to b$
7. $I \to Ia$
8. $I \to Ib$
9. $I \to I0$
10. $I \to I1$



文法和语言的歧义性

定义：若CFG G使某些符号串有两棵不同的语法分析树，则称该文法是歧义的。

例：算术表达式文法G_{exp}中，对句型$a*(a+a)$有下面两棵语法树：



定义：定义同样的语言可以有多个文法。如果上下文无关语言L的所有文法都是有歧义的，那么称语言L是固有歧义的。

如语言$L=\{a^ib^jc^k|i=j\ or\ j=k\}$是固有歧义的。

文法的化简

文法的简化步骤：

1. 消除ε-产生式
2. 消除单元产生式
3. 消除非产生的无用符号
4. 消除非可达的无用符号

文法的化简1：消除ε-产生式

定义：文法中形如$A \to \varepsilon$的产生式称为**ε-产生式**。如果变元$A \overset{*}{\Rightarrow} \varepsilon$，称$A$是**可空的**。

注意：对于CFL L，消除其文法中全部的ε-产生式后，得到的语言是$L - \{\varepsilon\}$。

- 确定“可空变元”：

1. 若$A \to \varepsilon$，则A是可空的；
2. 若$B \to \alpha$且α中的**每个符号**都是可空的，则B是可空的。

- 替换产生式：将含有可空变元的一条产生式$A \to X_1 X_2 \ldots X_n$，用一组产生式$A \to Y_1 Y_2 \ldots Y_n$代替，其中：
  1. 若X_i不是可空的，Y_i为X_i；
  2. 若X_i是可空的，Y_i为X_i或ε；
  3. 但Y_i不能全部都为ε。

文法的化简1：消除ε-产生式

例：消除CFG $G=(\{S,A,B\},\{a,b\},P,S)$的ε-产生式，其中$P$为$S\to AB$，$A\to AaA|\varepsilon$，$B\to BbB|\varepsilon$。

① 先确定全部可空的变元：$\{S,A,B\}$(A,B产生ε；$S\to AB$的右部变元只有A,B)

② 再替换全部带有可空符号的产生式(注意，每个可空变元有出现或不出现在新产生式中的两种情况)：

- $S\to AB$通过替换可空符号得到$S\to AB\mid A\mid B$
- $A\to AaA|\varepsilon$通过替换可空符号得到$A\to AaA\mid Aa\mid aA\mid a$
- $B\to BbB|\varepsilon$通过替换可空符号得到$B\to BbB\mid Bb\mid bB\mid b$

文法的化简2：消除单元产生式

- 确定单元对：如果有$A \overset{*}{\Rightarrow} B$，则称$[A,B]$为单元对

1. 若$A \to B \in P$，则$[A,B]$为单元对；
2. 若$[A,B]$和$[B,C]$都是单元对，则$[A,C]$是单元对。

- 消除单元产生式

1. 删除全部形为$A \to B$的单元产生式；
2. 对每个单元对$[A,B]$，将B的产生式复制给A。

文法的化简2：消除单元产生式

例：给定CFG $G=(\{S,A,B\},\{0,1\},P,S)$的 ε-产生式，其中$P$为 $S\to A|B|0S1$，$A\to 0A|0$，$B\to 1B|1$。

① 先确定单元产生式：$S\to A$，$S\to B$

② 再代入非单元产生式：

（1）非单元产生式：$S\to 0S1$，$A\to 0A\,|\,0$，$B\to 1B\,|\,1$

（2）非单元产生式：$A\to 0A|0$ 通过代入$S\to A$得到 $S\to 0A\,|\,0$

（3）非单元产生式：$B\to 1B\,|\,1$ 通过代入 $S\to B$ 得到$S\to 1B\,|\,1$

文法的化简3和4：消除无用符号

定义：CFG $G=(V,T,P,S)$，符号$X \in (V \cup T)$：

- 若$S \overset{*}{\Rightarrow} \alpha X \beta$，称$X$是可达的(reachable)
- 若$\alpha X \beta \overset{*}{\Rightarrow} w (w \in T^*)$，称$X$是产生的(generating)
- 若X同时是产生的和可达的，则称X是有用的，否则称X为无用符号。

计算“产生的”符号集

1. 每个T中的符号都是产生的
2. $A \to \alpha \in P$且α中的符号都是产生的，则A是产生的

计算“可达的”符号集

1. 符号S是可达的
2. $A \to \alpha \in P$且A是可达的，则α中的符号都是可达的

文法的化简3和4：消除无用符号

例：消除如下文法中的无用符号：$S \to AB|a$，$A \to b$

- **消除非产生的：**

“产生的”符号集为$\{a, b, S, A\}$（a, b是终结符；S产生a,A产生b);

消除B,得到$S \to a$,A $\to b$。

- **消除非可达的：**

“可达的”符号集为$\{S, a\}$ (S为开始符号，a可以由S到达)；

消除$\{b, A\}$，得到$S \to a$。

乔姆斯基范式（CNF）

定理：每个不带ε的CFL都可以由这样的CFG G定义，G中的每个产生式的形式都为$A \to BC$或$A \to a$，这里的A, B, C是变元，a是终结符。

CFG转换为CNF：

- 设文法不带无用符号，ε-产生式和单元产生式（任何文法都可以化简成这种形式）。
- 考虑文法每个形式为$A \to X_1 X_2 \cdots X_m (m \geq 2)$的产生式，若$X_i$为终结符$a$，则引进新变元$C_a$替换$X_i$并增加新产生式$C_a \to a$。
- 所有产生式的形式变为$A \to B_1 B_2 \cdots B_m (m \geq 2)$和$B_i$(即$C_a \to a$)。
- 对于$A \to B_1 B_2 \cdots B_m (m \geq 2)$，引入新变元$D_1, D_2, \cdots, D_{m-2}$，将其替换为一组级联产生式

$$A \to B_1 D_1,\ D_1 \to B_2 D_2, \cdots D_{m-2} \to B_{m-1} B_m$$

乔姆斯基范式（CNF）

例：CFG $G = (\{S, A, B\}, \{a, b\}, P, S)$，其中$P$为$S \to bA|aB$，$A \to bAA|aS|a$，$B \to aBB|bS|b$，为其设计等价的CNF文法。

① 替换终结符: $C_b \to b, C_a \to a$

$S \to bA|aB$ -> $S \to C_bA|C_aB$

$A \to bAA \mid aS \mid a$ -> $A \to C_bAA|C_aS|a$

$B \to aBB \mid bS \mid b$ -> $B \to C_aBB|C_bS|b$

② 对仅包含变元、长度大于2的产生式进行转换：

$A \to C_bAA$ -> $A \to C_bD_1, D_1 \to AA$

$B \to C_a\,BB$ -> $B \to C_a\,D_2, D_2 \to BB$

格雷巴赫范式（GNF）

定理：每个不带ε的CFL都可以由这样的CFG G定义，G中的每个产生式的形式都为$A \to a\alpha$，这里的A是变元，a是终结符，α是零或多个变元的串。

CFG转换为GNF：

考虑文法每个形式为$A \to X_1X_2...X_n$的产生式，若$X_i(i \geq 2)$为终结符a，则引进新变元C_a替换X_i并增加新产生式$C_a \to a$。此时，仅需处理X_1

X_1存在以下两种情况：

1. X_1可以转换为GNF
2. X_1不可转换为GNF，存在左递归

格雷巴赫范式（GNF）

例1：将以下文法转换为GNF：$S \to 01S1|00$

引入变元A，增加产生式$A \to 1$，$B \to 0$,则

$S \to 01S1|00$ -> $S \to 0ASA|0B$，$A \to 1$，$B \to 0$

例2：将以下文法转换为GNF：$S \to AB$，$A \to aA|bB|b$，$B \to b$

对$S \to AB$进行代入：

$$S \to aAB \mid bBB \mid bB$$
$$A \to aA \mid bB \mid b$$
$$B \to b$$

格雷巴赫范式（GNF）

存在左递归时：

- **直接左递归：形式为$A \to a\alpha$，若有A产生式：**

$$A \to A\alpha_1|A\alpha_2|\ldots|A\alpha_n|\beta_1|\beta_2|\ldots|\beta_m$$

其中，$\alpha_i \neq \varepsilon$，$\beta_j$不以$A$开始。

消除方法：引入新的变元B，并用如下产生式替换：

$$A \to \beta_1|\beta_2|\ldots|\beta_m|\beta_1 B|\beta_2 B|\ldots|\beta_m B$$

$$B \to \alpha_1|\alpha_2|\ldots|\alpha_n|\alpha_1 B|\alpha_2 B|\ldots|\alpha_n B$$

- **间接左递归：形式为**

$$A \to B\alpha|C \qquad B \to A\beta|D$$

派生过程可能产生$A \Rightarrow B\alpha \Rightarrow A\beta\alpha$。

消除方法：

① 将文法中变元重命名为$A_1, A_2, , , A_n$；

② 通过代入，令产生式都形如：

$$A_i \to A_j\alpha \text{ 或 } A_i \to a\alpha$$

但要求$i \leq j$；

③ 消除直接左递归$A_i \to A_i\beta$，再带入其他产生式。

格雷巴赫范式（GNF）

- **例：将下面的CFG转换成GNF。**

$S \to AB \quad A \to BS|b \quad B \to SA|a$

> •直接左递归：形式为$A \to a\alpha$，若有A产生式：
>
> $A \to A\alpha_1|A\alpha_2| \dots |A\alpha_n|\beta_1|\beta_2| \dots |\beta_m$
>
> 其中，$\alpha_i \neq \varepsilon$，$\beta_j$不以$A$开始。
>
> **消除方法：**引入新的变元B，并用如下产生式替换：
>
> $A \to \beta_1|\beta_2| \dots |\beta_m|\beta_1 B|\beta_2 B| \dots |\beta_m B$
>
> $B \to \alpha_1|\alpha_2| \dots |\alpha_n|\alpha_1 B|\alpha_2 B| \dots |\alpha_n B$

① 重命名变元，代换$i > j$的A_j，

$A_1 \to A_2A_3 \quad A_2 \to A_3A_1|b \quad A_3 \to A_1A_2|a \quad A_3 \to A_2A_3A_2|a \quad A_3 \to A_3\boldsymbol{A_1A_3A_2}|bA_3A_2|a$

β_1 β_2

② 消除直接左递归：

$A_1 \to A_2A_3 \quad A_2 \to A_3A_1|b \quad A_3 \to bA_3A_2|a|\ bA_3A_2B_1|aB_1 \quad B_1 \to A_1A_3A_2|A_1A_3A_2B_1$

③ 将A_3代入到A_2，A_2代入到A_1，A_1代入到B_1，

$A_2 \to bA_3A_2A_1|aA_1|bA_3A_2B_1A_1|aB_1A_1|b$

$A_1 \to bA_3A_2A_1A_3|aA_1A_3|bA_3A_2B_1A_1A_3|aB_1A_1A_3|bA_3$

$B_1 \to bA_3A_2A_1A_3A_3A_2|aA_1A_3A_3A_2|bA_3A_2B_1A_1A_3A_3A_2|aB_1A_1A_3A_3A_2|bA_3\ A_3A_2$

$bA_3A_2A_1A_3A_3A_2B_1|aA_1A_3A_3A_2B_1|bA_3A_2B_1A_1A_3A_3A_2B_1|aB_1A_1A_3A_3A_2B_1|bA_3\ A_3A_2B_1$